

College Management System Using Python and MySql

Abstract

This project deals with the various functioning in College management process. The main idea is to implement a proper process to system. In our existing system contains a many operations Student registration, fees, Marks, exam records, Sports and Events of the student etc. All these activity takeout systematically by administrator. In our model, it deals with the Operations in system. For example, when admin add all the forms in this system stored in the database. And display the details of student is perform by retrieving information from database table.

Introduction

As the college and information services community seeks to cope with new patterns of information provision, new technology, and changing financial circumstances, it is critical to gain new thinking across the profession. The innovative theory, and best organizational practice are all presented in the College Management System. The college management system provides knowledge about all aspects related to detailed information for students in order to follow them effectively. The college management system is one of the most tedious processes, which includes the regular updating of many files and records. When new students register, the administration has to issue a unique code for that student. For this, they have to keep track of many records like student details, address, phone number etc. and keep many records. Doing this manually takes a lot of time and effort. Hence, this can be facilitated by automation.

Existing System

The college is an ideal candidate for an IT solution, due to the vast amounts of data that it has to deal with on a daily basis. If the administration wants to search for a record of the student, they have to search for all records. If the administration wants to add a new student to the database, they must search for the subject or branch the student belongs to. If students want to know the financial situation or what is related to the fees, they must in the manual system come to the

college to find out the status. The whole process is now controlled automatically. This contributes to the fact that records of inquiries from students do not need to be kept on paper

Disadvantages

Maintaining the data in excel sheets and files is very hard to remember the file names in which the required data is feed. No easy access to the required queries. Data redundancy, inconsistency, a lot of human work need to be done to analyse the details present in the excel sheets. It leads to wastage of time. No timely response to the end-users.

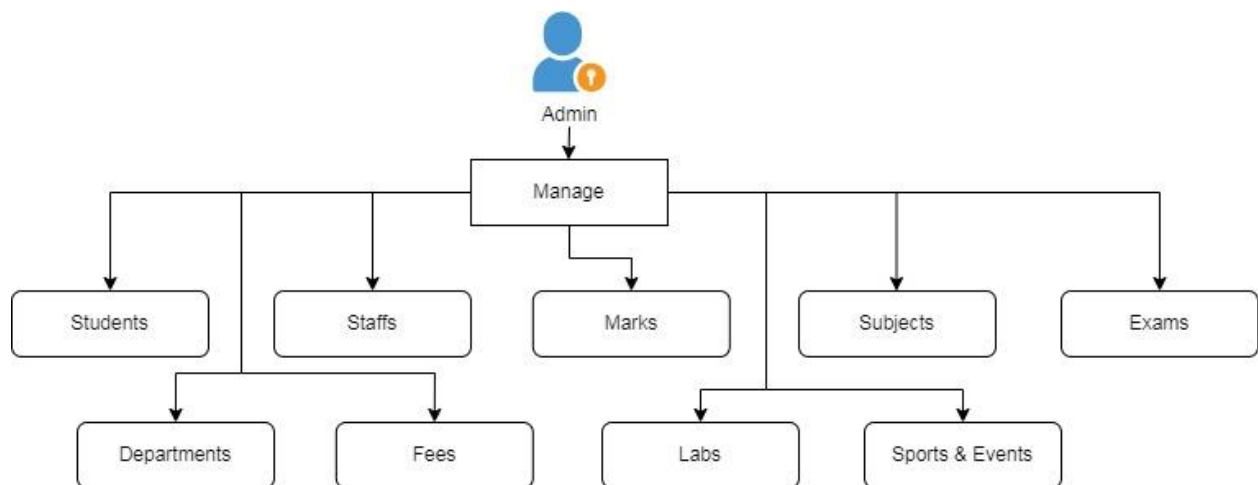
Proposed System

The proposed system is the one that is going to be automated so that it is easy to retrieve the responses from the system fastly and updating the details once the response or services are provided to the end-users upon their request without any difficulty and saves time.

Advantages

- Computerization of the details of the students.
- Maintenance time and cost are greatly reduced.
- Accurate information can be generated easily and quickly at different levels.

Architecture



Modules

- Login
- Student Module
- Staff Module
- Mark Module
- Subject Module
- Exam Module
- Department Module
- Sports and Events Module
- Laboratory Module
- Fee module

Module Description

Login

This module is used to authentication admin to provide ID and Password for Login, After Login Successful the system opens the application.

Student Module

The admin can add and manage the student details. The system will Store student Records in this system using Database.

Staff Module

This module is used to add staffs details and view staffs details. This detail consists of staff salary and contact details.

Mark Module

Mark module is to store the exam result of every student, it contains RegNo, Name and Marks

Subject Module

This module is used to store department wise Subjects and Staff allocation for all the subjects

Exam Module

In this module was storing the examination details like exam date, subject and other details.

Department Module

This module is to clustering the department using degree, these details has stored by the admin using various Programs

Sports and Events Module

The admin can add the sports details to this system and view these details it contains all Eligibility criteria for every sports. And also admin can store the event details like workshop, Seminar etc.

Laboratory Module

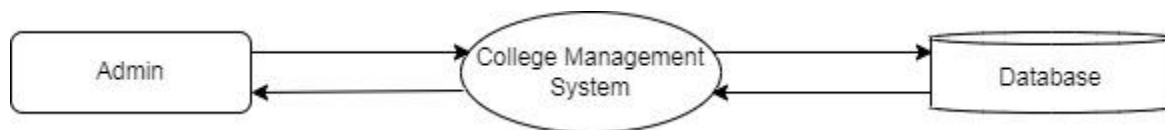
The laboratory module is to store the details of lab name, Equipments and departments in this system.

Fee module

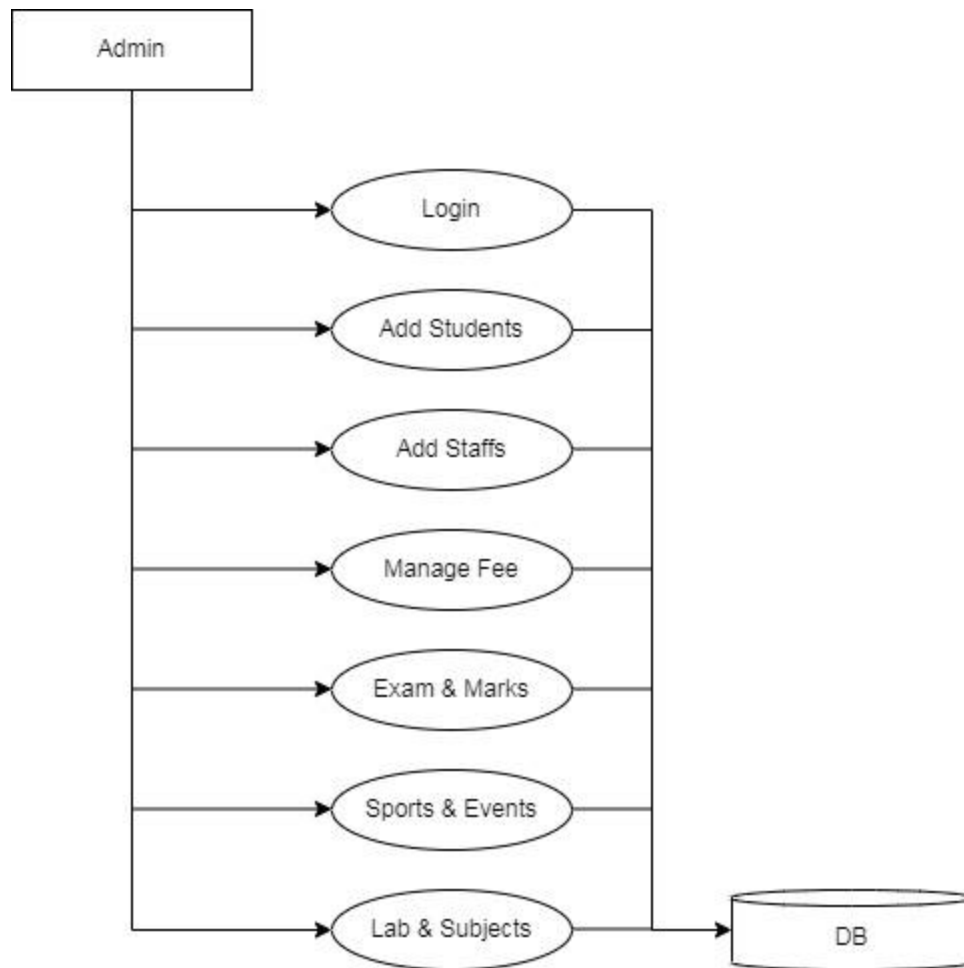
This module is used to Store and View the Fee Structure like tution fee, lab fee and other fee.

Data Flow Diagram

Level 1



Level 2



System Specification

Software Requirements:

Operating system : Windows 7 above
Coding Language : Python
IDE : IDLE (Python)

Hardware Requirements:

System : Pentium IV 2.4 GHz.
Hard Disk : 240 GB.
Monitor : 14' Colour Monitor.
Mouse : Optical Mouse.
Ram : 2 GB.

Software Description: Python

Python is a general purpose, dynamic, high-level and interpreted programming language. It supports Object Oriented programming approach to develop applications. It is simple and easy to learn and provides lots of high-level data structures.

Python is *easy to learn* yet powerful and versatile scripting language, which makes it attractive for Application Development.

Python's syntax and *dynamic typing* with its interpreted nature make it an ideal language for scripting and rapid application development.

Python supports *multiple programming pattern*, including object-oriented, imperative, and functional or procedural programming styles.

Python is not intended to work in a particular area, such as web programming. That is why it is known as *multipurpose* programming language because it can be used with web, enterprise, 3D CAD, etc.

We don't need to use data types to declare variable because it is *dynamically typed* so we can write `a=10` to assign an integer value in an integer variable.

Python makes the development and debugging *fast* because there is no compilation step included in Python development, and edit-test-debug cycle is very fast.

Python 2 vs. Python 3

In most of the programming languages, whenever a new version releases, it supports the features and syntax of the existing version of the language, therefore, it is easier for the projects to switch in the newer version. However, in the case of Python, the two versions Python 2 and Python 3 are very much different from each other.

A list of differences between Python 2 and Python 3 are given below:

1. Python 2 uses `print` as a statement and used as `print "something"` to print some string on the console. On the other hand, Python 3 uses `print` as a function and used as `print("something")` to print something on the console.
2. Python 2 uses the function `raw_input()` to accept the user's input. It returns the string representing the value, which is typed by the user. To convert it into the integer, we need to use the `int()` function in Python. On the other hand, Python 3 uses `input()` function which automatically interpreted the type of input entered by the user. However, we can cast this value to any type by using primitive functions (`int()`, `str()`, etc.).
3. In Python 2, the implicit string type is ASCII, whereas, in Python 3, the implicit string type is Unicode.
4. Python 3 doesn't contain the `xrange()` function of Python 2. The `xrange()` is the variant of `range()` function which returns a `xrange` object that works similar to Java iterator. The `range()` returns a list for example the function `range(0,3)` contains 0, 1, 2.
5. There is also a small change made in Exception handling in Python 3. It defines a keyword as which is necessary to be used. We will discuss it in Exception handling section of Python programming tutorial.

Python History

Python was invented by Guido van Rossum in 1991 at CWI in Netherland. The idea of Python programming language has taken from the ABC programming language or we can say that ABC is a predecessor of Python language.

There is also a fact behind the choosing name Python. Guido van Rossum was a fan of the popular BBC comedy show of that time, "Monty Python's Flying Circus". So he decided to pick the name Python for his newly created programming language.

Python has the vast community across the world and releases its version within the short period.

Python Concepts

Python provides many useful features to the programmer. These features make it most popular and widely used language. We have listed below few-essential feature of Python.

- Easy to use and Learn
- Expressive Language
- Interpreted Language
- Object-Oriented Language
- Open Source Language
- Extensible
- Learn Standard Library
- GUI Programming Support
- Integrated
- Embeddable
- Dynamic Memory Allocation
- Wide Range of Libraries and Frameworks

Where is Python used?

Python is a general-purpose, popular programming language and it is used in almost every technical field. The various areas of Python use are given below.

- Data Science
- Data Mining
- Desktop Applications
- Console-based Applications
- Mobile Applications
- Software Development
- Artificial Intelligence
- Web Applications
- Enterprise Applications

- 3D CAD Applications
- Machine Learning
- Computer Vision or Image Processing Applications.
- Speech Recognitions

Python Basic Syntax

There is no use of curly braces or semicolon in Python programming language. It is English-like language. But Python uses the indentation to define a block of code. Indentation is nothing but adding whitespace before the statement when it is needed. For example -

```
1. def func():  
2.     statement 1  
3.     statement 2  
4.     .....  
5.     .....  
6.     statement N
```

In the above example, the statements that are same level to right belong to the function. Generally, we can use four whitespaces to define indentation.

Python First Program

Unlike the other programming languages, Python provides the facility to execute the code using few lines. For example - Suppose we want to print the "Hello World" program in Java; it will take three lines to print it.

1. `public class HelloWorld {`
2. `public static void main(String[] args){`
3. `// Prints "Hello, World" to the terminal window.`
4. `System.out.println("Hello World");`
5. `}`
6. `}`

On the other hand, we can do this using one statement in Python.

1. `print("Hello World")`

Both programs will print the same result, but it takes only one statement without using a semicolon or curly braces in Python.

Python Popular Frameworks and Libraries

Python has wide range of libraries and frameworks widely used in various fields such as machine learning, artificial intelligence, web applications, etc. We define some popular frameworks and libraries of Python as follows.

- Web development (Server-side) - Django Flask, Pyramid, CherryPy
- GUIs based applications - Tk, PyGTK, PyQt, PyJs, etc.
- Machine Learning - TensorFlow, PyTorch, Scikit-learn, Matplotlib, Scipy, etc.
- Mathematics - Numpy, Pandas, etc.

Python print() Function

The print() function displays the given object to the standard output device (screen) or to the text stream file.

Unlike the other programming languages, Python print() function is most unique and versatile function.

The syntax of print() function is given below.

1. `print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`

Let's explain its parameters one by one.

- objects - An object is nothing but a statement that to be printed. The * sign represents that there can be multiple statements.
- sep - The sep parameter separates the print values. Default values is ' '.
- end - The end is printed at last in the statement.
- file - It must be an object with a write(string) method.
- flush - The stream or file is forcibly flushed if it is true. By default, its value is false.

Let's understand the following example.

Example - 1: Return a value

1. `print("Welcome to javaTpoint.")`
- 2.

3. `a = 10`
4. `# Two objects are passed in print() function`
5. `print("a =", a)`
- 6.
7. `b = a`
8. `# Three objects are passed in print function`
9. `print('a =', a, '= b')`

Output:

```
Welcome to javaTpoint.
```

```
a = 10
```

```
a = 10 = b
```

As we can see in the above output, the multiple objects can be printed in the single `print()` statement. We just need to use comma (,) to separate with each other.

Example - 2: Using `sep` and `end` argument

1. `a = 10`
2. `print("a =", a, sep='dddd', end='\n\n\n')`
3. `print("a =", a, sep='0', end='$$$$$')`

Output:

```
a =dddd10
```

```
a =010$$$$$
```

In the first print() statement, we use the sep and end arguments. The given object is printed just after the sep values. The value of end parameter printed at the last of given object. As we can see that, the second print() function printed the result after the three black lines.

Taking Input to the User

Python provides the input() function which is used to take input from the user. Let's understand the following example.

Example -

1. `name = input("Enter a name of student:")`
2. `print("The student name is: ", name)`

Output:

```
Enter a name of student: Devansh
The student name is:  Devansh
```

By default, the input() function takes the string input but what if we want to take other data types as an input.

If we want to take input as an integer number, we need to typecast the input() function into an integer.

For example -

Example -

1. `a = int(input("Enter first number: "))`
2. `b = int(input("Enter second number: "))`
- 3.
4. `print(a+b)`

Output:

```
Enter first number: 50
Enter second number: 100
150
```

We can take any type of values using `input()` function.

Python Operators

Operators are the symbols which perform various operations on Python objects. Python operators are the most essential to work with the Python data types. In addition, Python also provides identify membership and bitwise operators. We will learn all these operators with the suitable example in following tutorial.

- Python Operators

Python Conditional Statements

Conditional statements help us to execute a particular block for a particular condition. In this tutorial, we will learn how to use the conditional expression to execute a different block of statements. Python provides `if` and `else` keywords to set up logical conditions. The `elif` keyword is also used as conditional statement.

- Python `if..else` statement

Python Loops

Sometimes we may need to alter the flow of the program. The execution of a specific code may need to be repeated several numbers of times. For this purpose, the programming languages provide various types of loops capable of repeating some specific code several times. Consider the following tutorial to understand the statements in detail.

- Python Loops
- Python For Loop
- Python While Loop

Python Data Structures

Data structures are referred which can hold some data together or we say that they are used to store the data in organized way. Python provides built-in data structures such as list, tuple, dictionary, and set. We can perform complex tasks using data structures.

Python List

Python list holds the ordered collection of items. We can store a sequence of items in a list. Python list is mutable which means it can be modified after its creation. The items of lists are enclosed within the square bracket [] and separated by the comma. Let's see the example of list.

1. `L1 = ["John", 102, "USA"]`
2. `L2 = [1, 2, 3, 4, 5, 6]`

If we try to print the type of L1, L2, and L3 using `type()` function then it will come out to be a list.

1. `print(type(L1))`
2. `print(type(L2))`

Output:

```
<class 'list'>
<class 'list'>
```

To learn more about list, visit the following tutorial.

- [Python List](#)
- [Python List Functions](#)

Python Tuple

Python Tuple is used to store the sequence of immutable Python objects. The tuple is similar to lists since the value of the items stored in the list can be changed, whereas the tuple is immutable, and the value of the items stored in the tuple cannot be changed.

Tuple can be defined as follows

Example -

1. `tup = ("Apple", "Mango", "Orange", "Banana")`
2. `print(type(tup))`
3. `print(tup)`

Output:


```
<class 'tuple'>
('Apple', 'Mango', 'Orange', 'Banana')
```

If we try to add new to the tuple, it will throw an error.

Example -

1. `tup = ("Apple", "Mango" , "Orange" , "Banana")`
- 2.
3. `tup[2] = "Papaya"`
4. `print(tup)`

Output:

```
Traceback (most recent call last):
  File "C:/Users/DEVANSH SHARMA/PycharmProjects/Hello/gamewithturtle.py", line 3, in
    tup[2] = "Papaya"
TypeError: 'tuple' object does not support item assignment
```

The above program throws an error because tuples are immutable type. To learn more about tuple, visit the [Python Tuples](#).

- Python Tuple

Python String

Python string is a sequence of characters. It is a collection of the characters surrounded by single quotes, double quotes, or triple quotes. It can also define as collection of the Unicode characters. We can create a string as follows.

Example -

1. # Creating string using double quotes
2. str1 = "Hi Python"
3. print(str1)
4. # Creating string using single quotes
5. str1 = 'Hi Python'
6. print(str1)
7. # Creating string using triple quotes
8. str1 = """Hi Python"""
9. print(str1)

Output:

```
Hi Python
Hi Python
Hi Python
```

Python doesn't support the character data-type. A single character written as 'p' is treated as a string of length 1.

Strings are also immutable. We can't change after it is declared. To learn more about the string, visit the following tutorial.

- [Python Strings](#)
- [Python String Method](#)

Dictionaries

Python Dictionary is a most efficient data structure and used to store the large amount of data. It stores the data in the key-value pair format. Each value is stored corresponding to its key.

Keys must be a unique and value can be any type such as integer, list, tuple, etc.

It is a mutable type; we can reassign after its creation. Below is the example of creating dictionary in Python.

Example -

1. `employee = {"Name": "John", "Age": 29, "salary":250000,"Company":"GOOGLE"}`
2. `print(type(employee))`
3. `print("printing Employee data .... ")`
4. `print(employee)`

Output:

```
<class 'dict'>
Printing Employee data ....
{'Name': 'John', 'Age': 29, 'salary': 250000, 'Company': 'GOOGLE'}
```

The empty curly braces {} are used to create empty dictionary. To learn more, visit the complete tutorial of the dictionary.

- Python Dictionary
- Python Dictionary Methods

Python Sets

A Python set is a collection of unordered elements. Each element in set must be unique and immutable. Sets are mutable which means we can modify anytime throughout the program. Let's understand the example of creating set in Python.

Example -

1. # Creating Set
2. Month = {"January", "February", "March", "April", "May", "June", "July"}
3. print(Month)
4. print(type(Month))

Output:

```
{'March', 'July', 'April', 'May', 'June', 'February', 'January'}  
<class 'set'>
```

To get the more information about sets, visit the following resources.

- [Python Sets](#)
- [Python Set Methods](#)

Python Functional Programming

This section of Python tutorial defines some important tools related to functional programming such as lambda and recursive functions. These functions are very efficient in accomplishing the complex tasks. We define a few important functions, such as reduce, map, and filter. Python provides the functools module that includes various functional programming tools. Visit the following tutorial to learn more about functional programming.

- [Python Function](#)

- Python map() Function
- Python filter() Function
- Python reduce() Function
- Python functools Module
- Python Lambda Function

Python File I/O

Files are used to store data in a computer disk. In this tutorial, we explain the built-in file object of Python. We can open a file using Python script and perform various operations such as writing, reading, and appending. There are various ways of opening a file. We are explained with the relevant example. We will also learn to perform read/write operations on binary files.

- Python File I/O

Python Modules

Python modules are the program files that contain a Python code or functions. There are two types of module in the Python - User-define modules and built-in modules. A module that the user defines, or we can say that our Python code saved with .py extension, is treated as a user-define module.

Built-in modules are predefined modules of Python. To use the functionality of the modules, we need to import them into our current working program.

- Python Modules

Python Exceptions

An exception can be defined as an unusual condition in a program resulting in the interruption in the flow of the program.

Whenever an exception occurs, the program stops the execution, and thus the further code is not executed. Therefore, an exception is the run-time errors that are unable to handle to Python script. An exception is a Python object that represents an error.

- Python Exceptions

Python CSV

A csv stands for "comma separated values", which is defined as a simple file format that uses specific structuring to arrange tabular data. It stores tabular data such as spreadsheet or database in plain text and has a common format for data interchange. A csv file opens into the excel sheet, and the rows and columns data define the standard format. Visit the following tutorial to learn the CSV module in detail.

- Python Read CSV File
- Python Write CSV File

Python Sending Mail

We can send or read a mail using the Python script. Python's standard library modules are useful for handling various protocols such as PoP3 and IMAP. We will learn how to send a mail with the popular email service SMTP from a Python script.

- Python Sending Emails

Python Magic Methods

Python magic method is defined as the special method which adds "magic" to a class. It starts and ends with double underscores, for example, `__init__` or `__str__`.

The built-in classes define many magic methods. The `dir()` function can be used to see the number of magic methods inherited by a class. It has two prefixes and suffix underscores in the method name.

- Python Magic Methods

Python Oops Concepts

Everything in Python is treated as an object including integer values, floats, functions, classes, and none. Apart from that, Python supports all oriented concepts. Below is the brief introduction of oops concepts of Python.

- Classes and Objects - Python classes are the blueprint of the object. An object is a collection of data and method that act on the data.
- Inheritance - An inheritance is a technique where one class inherits the properties of other classes.
- Constructor - Python provides a special method `__init__()` which is known as a constructor. This method is automatically called when an object is instantiated.
- Data Member - A variable that holds data associated with a class and its objects.

To read the oops concept in detail, visit the following resources.

- [Python Oops Concepts](#)
- [Python Object and classes](#)
- [Python Constructor](#)
- [Python Inheritance](#)
- [Python Polymorphism](#)

Python Advance Topics

Python includes many advance and useful concepts that help the programmer to solve the complex tasks. These concepts are given below.

Python Iterator

An iterator is simply an object that can be iterated upon. It returns one object at a time. It can be implemented using the two special methods, `__iter__()` and `__next__()`.

To learn more about the iterators visit our [Python Iterators tutorial](#).

Python Generators

The Generators are an easiest way of creating Iterators. To learn more about, visit our [Python Generators tutorial](#).

Python Decorators

These are used to modify the behavior of the function. Decorators provide the flexibility to wrap another function to expand the working of wrapped function, without permanently modifying it.

To learn more about, visit the [Python Decorators tutorial](#).

Python Database Connections

We can use various databases along with Python. You can learn the full tutorial to visit below resources. Python DBI-API acclaims standard sets of functionality to be included in the database connectivity modules for respective RDBMS products. We explain all important database connectivity using Python DBI-API.

Python MySQL

Environment Setup

Database Connection

Creating New Database

Creating Tables

Insert Operation

Read Operation

Update Operation

Join Operation

Performing Transactions

Python MongoDB

Python MongoDB

Python SQLite

Python SQLite

Python CGI

Python CGI stands for "Common Gateway Interface", which is used to define how to exchange information between the webserver and a custom Python scripts. The Common Gateway Interface is a standard for external gateway programs to interface with the server, such as HTTP Servers. To learn more about Python CGI, visit the following tutorial.

- Python CGI

Prerequisite

Before learning Python, you must have the basic knowledge of programming concepts.

Audience

Our Python tutorial is designed to help beginners and professionals.

Problem

We assure that you will not find any problem in this Python tutorial. But if there is any mistake, please post the problem in contact form.

Pycharm

PyCharm is the most popular IDE used for Python scripting language. This chapter will give you an introduction to PyCharm and explains its features.

PyCharm offers some of the best features to its users and developers in the following aspects –

- Code completion and inspection
- Advanced debugging
- Support for web programming and frameworks such as Django and Flask

Features of PyCharm

Besides, a developer will find PyCharm comfortable to work with because of the features mentioned below –

Code Completion

PyCharm enables smoother code completion whether it is for built in or for an external package.

SQLAlchemy as Debugger

You can set a breakpoint, pause in the debugger and can see the SQL representation of the user expression for SQL Language code.

Git Visualization in Editor

When coding in Python, queries are normal for a developer. You can check the last commit easily in PyCharm as it has the blue sections that can define the difference between the last commit and the current one.

Code Coverage in Editor

You can run .py files outside PyCharm Editor as well marking it as code coverage details elsewhere in the project tree, in the summary section etc.

Package Management

All the installed packages are displayed with proper visual representation. This includes list of installed packages and the ability to search and add new packages.

Local History

Local History is always keeping track of the changes in a way that complements like Git. Local history in PyCharm gives complete details of what is needed to rollback and what is to be added.

Refactoring

Refactoring is the process of renaming one or more files at a time and PyCharm includes various shortcuts for a smooth refactoring process.

Testing

Testing is a process of executing a program with the intent of finding an error. Testing is a crucial element of software quality assurance and presents ultimate review of specification, design and coding. System Testing is an important phase. Testing represents an interesting anomaly for the software. Thus a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

A good test case is one that has a high probability of finding an as undiscovered error. A successful test is one that uncovers an as undiscovered error.

- Testing is a process of executing a program with the intent of finding an error Exhaustive testing is not possible
- To be most effective testing should be conducted by a independent third party.
- The primary objective for test case design is to derive a set of tests that has the highest livelihood for uncovering defects in software.
- To accomplish this objective two different categories of test case design techniques are used. They are

Unit Testing:

Unit testing is essential for the verification of the code produced during the coding phase and hence the goal is to test the internal logic of the modules. Using the detailed design description

as a guide, important paths are tested to uncover errors within the boundary of the modules. These tests were carried out during the programming stage itself. All units of Vienna SQL were successfully tested.

Integration Testing:

Integration testing focuses on unit tested modules and build the program structure that is dictated by the design phase.

System Testing:

System testing tests the integration of each module in the system. It also tests to find discrepancies between the system and its original objective, current specification and system documentation. The primary concern is the compatibility of individual modules. Entire system is working properly or not will be tested here, and specified path ODBC connection will correct or not, and giving output or not are tested here these verifications and validations are done by giving input values to the system and by comparing with expected output. Top-down testing implementing here.

Acceptance Testing:

This testing is done to verify the readiness of the system for the implementation. Acceptance testing begins when the system is complete. Its purpose is to provide the end user with the confidence that the system is ready for use. It involves planning and execution of functional tests, performance tests and stress tests in order to demonstrate that the implemented system satisfies its requirements.

Tools of special importance during acceptance testing include:

Test coverage Analyzer – records the control paths followed for each test case.

Timing Analyzer – also called a profiler, reports the time spent in various regions of the code are areas to concentrate on to improve system performance.

Coding standards – static analyzers and standard checkers are used to inspect code for deviations from standards and guidelines.

Test Cases:

Test cases are derived to ensure that all statements in the program have been executed at least once during testing and that all logical conditions have been executed. Using White-Box testing methods, the software engineer can drive test cases that

- Guarantee that logical decisions on their true and false sides.
- Exercise all logical decisions on their true and false sides.
- Execute all loops at their boundaries and within their operational bounds.
- Exercise internal data structure to assure their validity.
- The test case specification for system testing has to be submitted for review before system testing commences.

Functional And Non Functional Requirements

Non – Functional Requirements

A Non-functional requirement for the system may be:

- Emails should be sent with a latency of no greater than 12 hours from such an activity.

The functional requirement is describing the behavior of the system as it relates to the system's functionality. The non-functional requirement elaborates a performance characteristic of the system.

Typically non-functional requirements fall into areas such as:

- Accessibility
- Capacity, current and forecast
- Compliance
- Documentation
- Disaster recovery
- Efficiency
- Effectiveness

- Extensibility
- Fault tolerance
- Interoperability
- Maintainability
- Privacy
- Portability
- Quality
- Reliability
- Resilience
- Response time
- Robustness
- Scalability
- Security
- Stability
- Supportability
- Testability

Non-functional requirements are sometimes defined in terms of metrics (something that can be measured about the system) to make them more tangible. Non-functional requirements may also describe aspects of the system that don't relate to its execution, but rather to its evolution over time (e.g. maintainability, extensibility, documentation, etc).

Functional Requirements

1. Functional requirements specifies a function that a system or system component must be able to perform. It can be documented in various ways. The most common ones are written descriptions in documents, and use cases.

2. Use cases can be textual enumeration lists as well as diagrams, describing user actions. Each use case illustrates behavioural scenarios through one or more functional requirements. Often, though, an analyst will begin by eliciting a set of use cases, from which the analyst can derive the functional requirements that must be implemented to allow a user to perform each use case.
3. Functional requirements is what a system is supposed to accomplish. It may be
 - Calculations
 - Technical details
 - Data manipulation
 - Data processing
 - Other specific functionality
4. A typical functional requirement will contain a unique name and number, a brief summary, and a rationale. This information is used to help the reader understand why the requirement is needed, and to track the requirement through the development of the system.

Conclusion

The college management system is an application to provide an integrated inquiry regarding students and mainly for staffs who work in the college. This application software was module successfully and was also tested successfully by running test cases. It is easy to use and has the required options, which the user can use to perform the required operations.

References

www.w3schools.com

www.docs.python.org

www.javatpoint.com

Appendix

Source Code

Login

```
from tkinter import *
from tkinter import messagebox, Label
import mysql.connector
from PIL import ImageTk, Image
import os
```

```
mydb=mysql.connector.connect(
    host="localhost",
    user="root",
    passwd="good",
    database="mydbs"

)
mycursor=mydb.cursor(buffered=True)
```

```
#mycursor.execute('SELECT * FROM AI_login.AI_login')
```

```
login = Tk()
login.title('College Management System')
login.geometry('800x700')
img = ImageTk.PhotoImage(Image.open("back.jpg"))
panel = Label(login, image = img)
panel.pack(side = "bottom", fill = "both", expand = "yes")
login.resizable(0, 0)
```

```
def connect_database():
    # modify your SELECT query to select the correct columns
```



```

mycursor.execute('SELECT Name, Address FROM customers')
username = usernameEntry1.get()
password = passwordEntry2.get()
check_login = f"SELECT Name FROM customers WHERE Name = '{username}'"
check_password = f"SELECT Address FROM customers WHERE Address = '{password}'"
mycursor.execute(check_login)
username_result = mycursor.fetchall()
mycursor.execute(check_password)
password_result = mycursor.fetchall()
usernameU = username_result
passwordP = password_result
print(username)
print(password)
if usernameU and passwordP and username == usernameU[0][0] and password ==
passwordP[0][0]:
    #messagebox.showinfo(title="", message='Login successfully')
    #login.destroy()
    import Dashboard
else:
    messagebox.showerror(title='Error', message='Error, username/password is incorrect')

heading : Label = Label(login, text='College Management System', font=('Arial', 24),
fg='#8e44ad')
heading.place(x=230, y=70)
heading : Label = Label(login, text='Login Form', font=('Arial', 20), fg='red')

heading.place(x=350, y=120)

heading=Label(login, text='Username:', font=('Arial', 20))
heading.place(x=230, y=217)

```

```
usernameEntry1=Entry(login, font=('Arial', 16), width=18)
usernameEntry1.place(x=380,y=225)
```

```
heading=Label(login, text='Password:', font=('Arial', 20))
heading.place(x=235, y=275)
```

```
passwordEntry2=Entry(login, font=('Arial', 16), width=18)
passwordEntry2.place(x=380,y=283)
passwordEntry2.configure(show='*') loginButton=Button(login, text='Login', font=('Arial',
16),bg='light grey', width=12, height=1, command=connect_database)
loginButton.place(x=360, y=330)
login.mainloop()
```

Dashboard

```
from tkinter import *
```

```
ws = Tk()
ws.geometry('1500x2000')
ws.title('DashBoard-College Management System')
ws['bg']='#ecf0f1'
```

```
f = ("Times bold", 14)
```

```
def view_students():
    import Students
```

```
def view_staffs():
    import Staffs
```

```
def view_marks():
```

```
import Marks
```

```
def view_subjects():
```

```
    import Subjects
```

```
def view_exams():
```

```
    import Exams
```

```
def view_depts():
```

```
    import Depts
```

```
def view_sports():
```

```
    import Sports
```

```
def view_events():
```

```
    import Events
```

```
def view_fees():
```

```
    import Fees
```

```
def view_labs():
```

```
    import Labs
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
```

```
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='DashBoard', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
students=Button(ws, text='Students', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_students)
```

```
students.place(x=100, y=330)
```

```
staffs=Button(ws, text='Staffs', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_staffs)  
staffs.place(x=300, y=330)
```

```
marks=Button(ws, text='Marks', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_marks)  
marks.place(x=500, y=330)
```

```
subjects=Button(ws, text='Subjects', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_subjects)  
subjets.place(x=700, y=330)
```

```
exams=Button(ws, text='Exams', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_exams)  
exams.place(x=900, y=330)
```

```
depts=Button(ws, text='Departments', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_depts)  
depts.place(x=150, y=430)
```

```
sports=Button(ws, text='Sports', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_sports)  
sports.place(x=350, y=430)
```

```
events=Button(ws, text='Events', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_events)  
events.place(x=550, y=430)  
fees=Button(ws, text='Fees', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_fees)  
fees.place(x=750, y=430)
```

```
labs=Button(ws, text='Labs', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_labs)  
labs.place(x=950, y=430)
```

```
ws.mainloop()
```

Students

```
from tkinter import *  
from tkinter import messagebox  
import mysql.connector  
  
ws = Tk()  
ws.geometry('1300x700')  
ws.title('Students-College Management System')  
ws['bg']='#ecf0f1'  
f = ("Times bold", 14)  
  
def connect_database2():  
    fname1 = fname.get()  
    lname1 = lname.get()  
    email1 = email.get()  
    mobile1 = mobile.get()  
    dob1 = dob.get()  
    sclass1 = sclass.get()  
    regno1 = regno.get()  
    gender1 = vars.get()  
    gen=""  
    if gender1==1:  
        gen="Male"  
    else:  
        gen="Female"
```

```
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m  
ydbbs")
```

```
mycursor=mysqldb.cursor()
```

```
try:
```

```
    sql = "INSERT INTO student (FName, LName, Email, Mobile, DOB, Gender, SClass,  
RegNo) VALUES (%s, %s, %s, %s, %s, %s, %s,%s)"
```

```
    val = (fname1,lname1,email1,mobile1,dob1,gen,sclass1,regno1)
```

```
    mycursor.execute(sql, val)
```

```
    mysqldb.commit()
```

```
    messagebox.showinfo("Student","Add Data Successfully")
```

```
except Exception as e:
```

```
    print(e)
```

```
    mysqldb.rollback()
```

```
    mysqldb.close()
```

```
def view_stud():
```

```
    import Viewstud
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_stud)
```

```
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
```

```
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='Student Details', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='First Name:', font=('Arial', 20))
heading.place(x=200, y=200)
```

```
fname=Entry(ws, font=('Arial', 16), width=18)
fname.place(x=350,y=200)
```

```
heading=Label(ws, text='Last Name:', font=('Arial', 20))
heading.place(x=650, y=200)
```

```
lname=Entry(ws, font=('Arial', 16), width=18)
lname.place(x=800,y=200)
```

```
heading=Label(ws, text='EmailID:', font=('Arial', 20))
heading.place(x=200, y=300)
```

```
email=Entry(ws, font=('Arial', 16), width=18)
email.place(x=350,y=300)
```

```
heading=Label(ws, text='Mobile:', font=('Arial', 20))
heading.place(x=650, y=300)
```

```
mobile=Entry(ws, font=('Arial', 16), width=18)
mobile.place(x=800,y=300)
```

```
heading=Label(ws, text='DOB:', font=('Arial', 20))
heading.place(x=200, y=400)
```

```
dob=Entry(ws, font=('Arial', 16), width=18)
dob.place(x=350,y=400)
```

```
heading=Label(ws, text='Gender:', font=('Arial', 20))
```

```
heading.place(x=650, y=400)
```

```
#mobile2=Entry(ws, font=('Arial', 16), width=18)
```

```
vars = IntVar()
```

```
Radiobutton(ws, text="Male", font=('Arial', 16), padx=5,variable=vars, value=1).place(x=800,  
y=400)
```

```
Radiobutton(ws, text="Female", font=('Arial', 16), padx=10,variable=vars,  
value=2).place(x=900,y=400)
```

```
#Radiobutton(base, text="others", padx=15, variable=vars, value=3).place(x=310,y=240)
```

```
heading=Label(ws, text='Class:', font=('Arial', 20))
```

```
heading.place(x=200, y=500)
```

```
sclass=Entry(ws, font=('Arial', 16), width=18)
```

```
sclass.place(x=350,y=500)
```

```
heading=Label(ws, text='Reg No:', font=('Arial', 20))
```

```
heading.place(x=650, y=500)
```

```
regno=Entry(ws, font=('Arial', 16), width=18)
```

```
regno.place(x=800,y=500)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=connect_database2)
```

```
RegButton.place(x=500, y=600)
```

```
ws.mainloop()
```

View students


```

import tkinter as tk
from tkinter import ttk
import mysql.connector

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
    database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT      FName,LName,Email,Mobile,DOB,Gender,SClass,RegNo
FROM student")
    records = mycursor.fetchall()
    print(records)

    for i, (FName,LName,Email,Mobile,DOB,Gender,SClass,RegNo) in enumerate(records,
start=1):
        listBox.insert("",                                "end",
values=(FName,LName,Email,Mobile,DOB,Gender,SClass,RegNo))
        mysqldb.close()

root = tk.Tk()
root.title("Student Records")
label = tk.Label(root, text="Student Records", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('FName','LName','Email','Mobile','DOB','Gender','SClass','RegNo')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)

```

```
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()
```

Staffs

```
from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Staffs-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    name1 = name.get()
    qual1 = qual.get()
    email1 = email.get()
    mobile1 = mobile.get()
    salary1 = salary.get()

    gender1 = vars.get()
    gen=""
    if gender1==1:
        gen="Male"
    else:
        gen="Female"
```

```
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m  
ydbbs")
```

```
mycursor=mysqldb.cursor()
```

```
try:
```

```
    sql = "INSERT INTO staff (Name, Qualification, Email, Mobile, Salary, Gender) VALUES  
(%s, %s, %s, %s, %s, %s)"
```

```
    val = (name1,qual1,email1,mobile1,salary1,gen)
```

```
    mycursor.execute(sql, val)
```

```
    mysqldb.commit()
```

```
    messagebox.showinfo("Staff","Add Data Successfully")
```

```
except Exception as e:
```

```
    print(e)
```

```
    mysqldb.rollback()
```

```
    mysqldb.close()
```

```
def view_staff():
```

```
    import Viewstaff
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_staff)
```

```
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')  
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='Staff Details', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Name:', font=('Arial', 20))
```

```
heading.place(x=200, y=200)
```

```
name=Entry(ws, font=('Arial', 16), width=18)
```

```
name.place(x=350,y=200)
```

```
heading=Label(ws, text='Qualification:', font=('Arial', 20))
```

```
heading.place(x=650, y=200)
```

```
qual=Entry(ws, font=('Arial', 16), width=18)
```

```
qual.place(x=800,y=200)
```

```
heading=Label(ws, text='EmailID:', font=('Arial', 20))
```

```
heading.place(x=200, y=300)
```

```
email=Entry(ws, font=('Arial', 16), width=18)
```

```
email.place(x=350,y=300)
```

```
heading=Label(ws, text='Mobile:', font=('Arial', 20))
```

```
heading.place(x=650, y=300)
```

```
mobile=Entry(ws, font=('Arial', 16), width=18)
```

```
mobile.place(x=800,y=300)
```

```
heading=Label(ws, text='Salary:', font=('Arial', 20))
```

```
heading.place(x=200, y=400)
```

```
salary=Entry(ws, font=('Arial', 16), width=18)
```

```
salary.place(x=350,y=400)
```

```
heading=Label(ws, text='Gender:', font=('Arial', 20))
```

```
heading.place(x=650, y=400)
```

```
#mobile2=Entry(ws, font=('Arial', 16), width=18)
vars = IntVar()
Radiobutton(ws, text="Male", font=('Arial', 16), padx=5,variable=vars, value=1).place(x=800,
y=400)
Radiobutton(ws, text="Female", font=('Arial', 16), padx=10,variable=vars,
value=2).place(x=900,y=400)
#Radiobutton(base, text="others", padx=15, variable=vars, value=3).place(x=310,y=240)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=12, height=1,
command=connect_database2)
```

```
RegButton.place(x=500, y=500)
```

```
ws.mainloop()
```

View Staff

```
import tkinter as tk
from tkinter import ttk
import mysql.connector

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT Name,Qualification,Email,Mobile,Salary,Gender FROM
staff")
    records = mycursor.fetchall()
```

```
print(records)
```

```
for i, (Name,Qualification,Email,Mobile,Salary,Gender) in enumerate(records, start=1):  
    listBox.insert("", "end", values=(Name,Qualification,Email,Mobile,Salary,Gender))  
mysqldb.close()
```

```
root = tk.Tk()
```

```
root.title("Staff Records")
```

```
label = tk.Label(root, text="Staff Records", font=("Arial",24)).grid(row=0, columnspan=3)
```

```
cols = ('Name','Qualification','Email','Mobile','Salary','Gender')
```

```
listBox = ttk.Treeview(root, columns=cols, show='headings')
```

```
for col in cols:
```

```
    listBox.heading(col, text=col)
```

```
    listBox.grid(row=1, column=0, columnspan=1)
```

```
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=6, column=1)
```

```
show()
```

```
root.mainloop()
```

Marks

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
import mysql.connector
```

```
ws = Tk()
```

```
ws.geometry('1300x700')
```

```
ws.title('Marks-College Management System')
```

```
ws['bg']='#ecf0f1'
```

```
f = ("Times bold", 14)
```

```
def connect_database2():
```

```
    regno1 = regno.get()
```

```
    name1 = name.get()
```

```
    s11 = s1.get()
```

```
    s21 = s2.get()
```

```
    s31 = s3.get()
```

```
    s41 = s4.get()
```

```
    s51 = s5.get()
```

```
    s61 = s6.get()
```

```
    num1=int(s11)
```

```
    num2=int(s21)
```

```
    num3=int(s31)
```

```
    num4=int(s41)
```

```
    num5=int(s51)
```

```
    num6=int(s61)
```

```
    t=num1+num2+num3+num4+num5+num6
```

```
mysqlldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m  
ydbbs")
```

```
    mycursor=mysqlldb.cursor()
```

```
    try:
```

```
        sql = "INSERT INTO marks (RegNo, Name, S1, S2, S3, S4, S5, S6, Total) VALUES (%s,  
%s, %s, %s, %s, %s, %s, %s)"
```

```
        val = (regno1,name1,s11,s21,s31,s41,s51,s61,t)
```

```
        mycursor.execute(sql, val)
```

```
        mysqlldb.commit()
```

```
        messagebox.showinfo("Marks", "Add Data Successfully")
```

```
    except Exception as e:
```

```
print(e)
mysqldb.rollback()
mysqldb.close()
```

```
def view_mark():
    import Viewmark
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_mark)
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Marks Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Reg No:', font=('Arial', 20))
heading.place(x=200, y=200)
```

```
regno=Entry(ws, font=('Arial', 16), width=18)
regno.place(x=350,y=200)
```

```
heading=Label(ws, text='Name:', font=('Arial', 20))
heading.place(x=650, y=200)
```

```
name=Entry(ws, font=('Arial', 16), width=18)
name.place(x=800,y=200)
```

```
heading=Label(ws, text='Subject 1:', font=('Arial', 20))
heading.place(x=200, y=300)
```



```
s1=Entry(ws, font=('Arial', 16), width=18)
```

```
s1.place(x=350,y=300)
```

```
heading=Label(ws, text='Subject 2:', font=('Arial', 20))
```

```
heading.place(x=650, y=300)
```

```
s2=Entry(ws, font=('Arial', 16), width=18)
```

```
s2.place(x=800,y=300)
```

```
heading=Label(ws, text='Subject 3:', font=('Arial', 20))
```

```
heading.place(x=200, y=400)
```

```
s3=Entry(ws, font=('Arial', 16), width=18)
```

```
s3.place(x=350,y=400)
```

```
heading=Label(ws, text='Subject 4:', font=('Arial', 20))
```

```
heading.place(x=650, y=400)
```

```
s4=Entry(ws, font=('Arial', 16), width=18)
```

```
s4.place(x=800,y=400)
```

```
heading=Label(ws, text='Subject 5:', font=('Arial', 20))
```

```
heading.place(x=200, y=500)
```

```
s5=Entry(ws, font=('Arial', 16), width=18)
```

```
s5.place(x=350,y=500)
```

```
heading=Label(ws, text='Subject 6', font=('Arial', 20))
```

```
heading.place(x=650, y=500)
```

```
s6=Entry(ws, font=('Arial', 16), width=18)
```

```
s6.place(x=800,y=500)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=connect_database2)
```

```
RegButton.place(x=500, y=600)
```

```
ws.mainloop()
```

View Marks

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",  
database="mydbs")
```

```
    mycursor = mysqldb.cursor()
```

```
    mycursor.execute("SELECT RegNo,Name,S1,S2,S3,S4,S5,S6,Total FROM marks")
```

```
    records = mycursor.fetchall()
```

```
    print(records)
```

```
    for i, (RegNo,Name,S1,S2,S3,S4,S5,S6,Total) in enumerate(records, start=1):
```

```
        listBox.insert("", "end", values=(RegNo,Name,S1,S2,S3,S4,S5,S6,Total))
```

```
    mysqldb.close()
```

```
root = tk.Tk()
```

```

root.title("Mark Details")
label = tk.Label(root, text="View Mark Details", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('RegNo','Name','S1','S2','S3','S4','S5','S6','Total')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Subjects

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Subjects-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    subject1 = subject.get()
    dept1 = dept.get()
    staff1 = staff.get()

    no=0
    sno=no

```

```
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m  
ydbbs")
```

```
mycursor=mysqldb.cursor()
```

```
try:
```

```
    sql = "INSERT INTO subjects (SNo, Subject, Department, Staff) VALUES (%s, %s, %s,  
%s)"
```

```
    val = (sno,subject1,dept1,staff1)
```

```
    mycursor.execute(sql, val)
```

```
    mysqldb.commit()
```

```
    messagebox.showinfo("Subjcts","Subject is Added")
```

```
except Exception as e:
```

```
    print(e)
```

```
    mysqldb.rollback()
```

```
    mysqldb.close()
```

```
def view_subject():
```

```
    import Viewsobject
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_subject)
```

```
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
```

```
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='Subjects Details', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Subject:', font=('Arial', 20))
heading.place(x=350, y=200)
```

```
subject=Entry(ws, font=('Arial', 16), width=18)
subject.place(x=550,y=200)
```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
heading.place(x=350, y=300)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
dept.place(x=550,y=300)
```

```
heading=Label(ws, text='Staff:', font=('Arial', 20))
heading.place(x=350, y=400)
```

```
staff=Entry(ws, font=('Arial', 16), width=18)
staff.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
```

```
RegButton.place(x=550, y=450)
```

```
ws.mainloop()
```

View Subjects

```
import tkinter as tk
from tkinter import ttk
import mysql.connector

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
    database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT SNo,Subject,Department,Staff FROM subjects")
    records = mycursor.fetchall()
    print(records)

    for i, (SNo,Subject,Department,Staff) in enumerate(records, start=1):
        listBox.insert("", "end", values=(SNo,Subject,Department,Staff))
    mysqldb.close()

root = tk.Tk()
root.title("Subject Records")
label = tk.Label(root, text="View Subject Details", font=("Arial",24)).grid(row=0,
columnspan=3)

cols = ('SNo','Subject','Department','Staff')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
```

```
root.mainloop()
```

Exams

```
from tkinter import *  
from tkinter import messagebox  
import mysql.connector  
from tkcalendar import Calendar, DateEntry
```

```
ws = Tk()  
ws.geometry('1300x700')  
ws.title('Exams-College Management System')  
ws['bg']='#ecf0f1'  
f = ("Times bold", 14)
```

```
def connect_database2():  
    ed1 = ed.get()  
    dept1 = dept.get()  
    subject1 = subject.get()  
    sem1 = clicked.get()
```

```
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m  
ydbbs")  
mycursor=mysqldb.cursor()
```

```
try:  
    sql = "INSERT INTO exams (EDate, Department, Subject, Semester) VALUES (%s, %s,  
%s, %s)"  
    val = (ed1,dept1,subject1,sem1)
```

```
mycursor.execute(sql, val)
mysqldb.commit()
messagebox.showinfo("Exam", "Success")
except Exception as e:
```

```
print(e)
mysqldb.rollback()
mysqldb.close()
```

```
options = [
    "1",
    "2",
    "3",
    "4",
    "5",
    "6",

]
```

```
clicked = StringVar()
clicked.set( "Select Sem" )
```

```
def view_exam():
    import Viewexam
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_exam)
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
```



```
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Exam Date & Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Date:', font=('Arial', 20))
heading.place(x=350, y=200)
```

```
ed = DateEntry(ws, font=('Arial', 16), width=18)
ed.place(x=550,y=200)
```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
heading.place(x=350, y=300)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
dept.place(x=550,y=300)
```

```
heading=Label(ws, text='Subject:', font=('Arial', 20))
heading.place(x=350, y=400)
```

```
subject=Entry(ws, font=('Arial', 16), width=18)
subject.place(x=550,y=400)
```

```
heading=Label(ws, text='Semester:', font=('Arial', 20))
heading.place(x=350, y=500)
```

```
#subject=Entry(ws, font=('Arial', 16), width=18)
sem = OptionMenu(ws , clicked , *options)
sem.place(x=550,y=500)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
```

```
RegButton.place(x=550, y=550)
```

```
ws.mainloop()
```

View Exams

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
```

```
    mycursor = mysqldb.cursor()
```

```
    mycursor.execute("SELECT EDate,Department,Subject,Semester FROM exams")
```

```
    records = mycursor.fetchall()
```

```
    print(records)
```

```
    for i, (EDate,Department,Subject,Semester) in enumerate(records, start=1):
```

```
        listBox.insert("", "end", values=(EDate,Department,Subject,Semester))
```

```
    mysqldb.close()
```

```
root = tk.Tk()
```

```
root.title("Exam Records")
```

```
label = tk.Label(root, text="View Exam Details", font=("Arial",24)).grid(row=0, columnspan=3)
```

```

cols = ('EDate','Department','Subject','Semester')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Department

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Department-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    degree1 = degree.get()
    dept1 = dept.get()
    desc1 = desc.get("1.0", "end-1c")

    no=0
    sno=no

```

```
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m  
ydbbs")
```

```
mycursor=mysqldb.cursor()
```

```
try:
```

```
    sql = "INSERT INTO depts (SNo, Degree, Department, Description) VALUES (%s, %s,  
%s, %s)"
```

```
    val = (sno,degree1,dept1,desc1)
```

```
    mycursor.execute(sql, val)
```

```
    mysqldb.commit()
```

```
    messagebox.showinfo("Department", "Success")
```

```
except Exception as e:
```

```
    print(e)
```

```
    mysqldb.rollback()
```

```
    mysqldb.close()
```

```
def view_dept():
```

```
    import Viewdept
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_dept)
```

```
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')  
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='Department Details', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Degree:', font=('Arial', 20))
```

```
heading.place(x=350, y=200)
```

```
degree=Entry(ws, font=('Arial', 16), width=18)
```

```
degree.place(x=550,y=200)
```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
```

```
heading.place(x=350, y=300)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
```

```
dept.place(x=550,y=300)
```

```
heading=Label(ws, text='Description:', font=('Arial', 20))
```

```
heading.place(x=350, y=400)
```

```
desc=Text(ws, height = 5, width = 52)
```

```
desc.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,  
command=connect_database2)
```

```
RegButton.place(x=550, y=500)
```

```
ws.mainloop()
```

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
    database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT SNo,Degree,Department,Description FROM depts")
    records = mycursor.fetchall()
    print(records)

    for i, (SNo,Degree,Department,Description) in enumerate(records, start=1):
        listBox.insert("", "end", values=(SNo,Degree,Department,Description))
        mysqldb.close()

root = tk.Tk()
root.title("Department Details")
label = tk.Label(root, text="View Exam Details", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('SNo','Degree','Department','Description')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Sports

```

from tkinter import *
from tkinter import messagebox

```

```
import mysql.connector
```

```
ws = Tk()
```

```
ws.geometry('1300x700')
```

```
ws.title('Sports-College Management System')
```

```
ws['bg']='#ecf0f1'
```

```
f = ("Times bold", 14)
```

```
def connect_database2():
```

```
    sports1 = sports.get()
```

```
    age1 = age.get()
```

```
    height1=height.get()
```

```
    weight1=weight.get()
```

```
    desc1 = desc.get("1.0", "end-1c")
```

```
    no=0
```

```
    sno=no
```

```
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="mydbs")
```

```
    mycursor=mysqldb.cursor()
```

```
    try:
```

```
        sql = "INSERT INTO sports (Sports, Age, Height, Weight, Description) VALUES (%s, %s, %s, %s, %s)"
```

```
        val = (sports1,age1,height1,weight1,desc1)
```

```
        mycursor.execute(sql, val)
```

```
        mysqldb.commit()
```

```
        messagebox.showinfo("Sports","Success")
```

```
except Exception as e:
```

```
    print(e)
```

```
    mysqldb.rollback()
```

```
    mysqldb.close()
```

```
def view_sport():
```

```
    import Viewsport
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_sport)
```

```
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
```

```
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='Sports Details', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Sports:', font=('Arial', 20))
```

```
heading.place(x=350, y=200)
```

```
sports=Entry(ws, font=('Arial', 16), width=18)
```

```
sports.place(x=550,y=200)
```

```
heading=Label(ws, text='Age:', font=('Arial', 20))
```

```
heading.place(x=350, y=250)
```

```
age=Entry(ws, font=('Arial', 16), width=18)
```

```
age.place(x=550,y=250)
```

```
heading=Label(ws, text='Hight:', font=('Arial', 20))
```



```
heading.place(x=350, y=300)
```

```
height=Entry(ws, font=('Arial', 16), width=18)
```

```
height.place(x=550,y=300)
```

```
heading=Label(ws, text='Weight:', font=('Arial', 20))
```

```
heading.place(x=350, y=350)
```

```
weight=Entry(ws, font=('Arial', 16), width=18)
```

```
weight.place(x=550,y=350)
```

```
heading=Label(ws, text='Description:', font=('Arial', 20))
```

```
heading.place(x=350, y=400)
```

```
desc=Text(ws, height = 5, width = 52)
```

```
desc.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,  
command=connect_database2)
```

```
RegButton.place(x=550, y=500)
```

```
ws.mainloop()
```

View Sports

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```

mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
mycursor = mysqldb.cursor()
mycursor.execute("SELECT Sports,Age,Height,Weight,Description FROM sports")
records = mycursor.fetchall()
print(records)

```

```

for i, (Sports,Age,Height,Weight,Description) in enumerate(records, start=1):
    listBox.insert("", "end", values=(Sports,Age,Height,Weight,Description))
mysqldb.close()

```

```

root = tk.Tk()
root.title("Sports Records")
label = tk.Label(root, text="View Sports Details", font=("Arial",24)).grid(row=0,
columnspan=3)

```

```

cols = ('Sports','Age','Height','Weight','Description')
listBox = ttk.Treeview(root, columns=cols, show='headings')

```

```

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Events

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

```

```
from tkcalendar import Calendar, DateEntry
```

```
ws = Tk()
```

```
ws.geometry('1300x700')
```

```
ws.title('Events-College Management System')
```

```
ws['bg']='#ecf0f1'
```

```
f = ("Times bold", 14)
```

```
def connect_database2():
```

```
    ed1 = ed.get()
```

```
    en1 = en.get()
```

```
    place1 = place.get()
```

```
    desc1 = desc.get("1.0", "end-1c")
```

```
mysqlldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m  
ydbbs")
```

```
    mycursor=mysqlldb.cursor()
```

```
    try:
```

```
        sql = "INSERT INTO events (EDate, Event, Place, Description) VALUES (%s, %s, %s,  
%s)"
```

```
        val = (ed1,en1,place1,desc1)
```

```
        mycursor.execute(sql, val)
```

```
        mysqlldb.commit()
```

```
        messagebox.showinfo("Events","Success")
```

```
    except Exception as e:
```

```
        print(e)
```

```
mysqldb.rollback()
mysqldb.close()
```

```
def view_event():
    import Viewevent
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_event)
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Event Date & Place', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Date:', font=('Arial', 20))
heading.place(x=350, y=200)
```

```
ed = DateEntry(ws, font=('Arial', 16), width=18)
ed.place(x=550,y=200)
```

```
heading=Label(ws, text='EventName:', font=('Arial', 20))
heading.place(x=350, y=250)
```

```
en=Entry(ws, font=('Arial', 16), width=18)
en.place(x=550,y=250)
```

```
heading=Label(ws, text='Place:', font=('Arial', 20))
heading.place(x=350, y=300)
```

```
place=Entry(ws, font=('Arial', 16), width=18)
place.place(x=550,y=300)
```

```
heading=Label(ws, text='Description:', font=('Arial', 20))
heading.place(x=350, y=350)
```

```
desc=Text(ws, height = 5, width = 52)
desc.place(x=550,y=350)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
```

```
RegButton.place(x=550, y=450)
```

```
ws.mainloop()
```

View Events

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
```

```
    mycursor = mysqldb.cursor()
```

```
    mycursor.execute("SELECT EDate,Event,Place,Description FROM events")
```

```
    records = mycursor.fetchall()
```

```
    print(records)
```

```

for i, (EDate,Event,Place,Description) in enumerate(records, start=1):
    listBox.insert("", "end", values=(EDate,Event,Place,Description))
mysqldb.close()

```

```

root = tk.Tk()
root.title("Event Records")
label = tk.Label(root, text="View Event Details", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('EDate','Event','Place','Description')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Fees

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Fees Details-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

```

```
def connect_database2():
```

```
    degree1 = degree.get()
```

```
    dept1 = dept.get()
```

```
    tfee1 = tfee.get()
```

```
    lfee1 = lfee.get()
```

```
    sfee1 = sfee.get()
```

```
    tf = int(tfee1)
```

```
    lf = int(lfee1)
```

```
    sf = int(sfee1)
```

```
    t = tf+lf+sf
```

```
    no=0
```

```
    sno=no
```

```
mysqlldb=mysql.connector.connect(host="localhost",user="root",password="good",database="mydbs")
```

```
    mycursor=mysqlldb.cursor()
```

```
    try:
```

```
        sql = "INSERT INTO fees (SNo, Degree, Department, TutionFee, LabFee, SFee, Total)
```

```
VALUES (%s, %s, %s, %s, %s, %s, %s)"
```

```
        val = (sno,degree1,dept1,tfee1,lfee1,sfee1,t)
```

```
        mycursor.execute(sql, val)
```

```
        mysqlldb.commit()
```

```
        messagebox.showinfo("Fees","Success")
```

```
    except Exception as e:
```

```
        print(e)
```

```
mysqldb.rollback()
mysqldb.close()
```

```
def view_fee():
    import Viewfee
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_fee)
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Fee Details / Year', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Degree:', font=('Arial', 20))
heading.place(x=350, y=200)
```

```
degree=Entry(ws, font=('Arial', 16), width=18)
degree.place(x=550,y=200)
```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
heading.place(x=350, y=250)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
dept.place(x=550,y=250)
```

```
heading=Label(ws, text='TutionFee:', font=('Arial', 20))
```



```
heading.place(x=350, y=300)
```

```
tfee=Entry(ws, font=('Arial', 16), width=18)
```

```
tfee.place(x=550,y=300)
```

```
heading=Label(ws, text='LabFee:', font=('Arial', 20))
```

```
heading.place(x=350, y=350)
```

```
lfee=Entry(ws, font=('Arial', 16), width=18)
```

```
lfee.place(x=550,y=350)
```

```
heading=Label(ws, text='Stationary:', font=('Arial', 20))
```

```
heading.place(x=350, y=400)
```

```
sfee=Entry(ws, font=('Arial', 16), width=18)
```

```
sfee.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
```

```
command=connect_database2)
```

```
RegButton.place(x=550, y=450)
```

```
ws.mainloop()
```

View Fees Details

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT SNo,Degree,Department,TutionFee,LabFee,SFee,Total FROM
fees")
    records = mycursor.fetchall()
    print(records)

    for i, (SNo,Degree,Department,TutionFee,LabFee,SFee,Total) in enumerate(records,
start=1):
        listBox.insert("", "end", values=(SNo,Degree,Department,TutionFee,LabFee,SFee,Total))
    mysqldb.close()

root = tk.Tk()
root.title("Fees Details")
label = tk.Label(root, text="View Fee Details", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('SNo','Degree','Department','TutionFee','LabFee','SFee','Total')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Labs

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Laboratory-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    ln1 = ln.get()
    dept1 = dept.get()
    desc1 = desc.get("1.0", "end-1c")

    no=0
    sno=no

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="m
ydb")
mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO labs (SNo, LabName, Department, Description) VALUES (%s, %s,
%s, %s)"
    val = (sno,ln1,dept1,desc1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Laboratory","Add Lab Details")

```

```
except Exception as e:
```

```
    print(e)
```

```
    mysqldb.rollback()
```

```
    mysqldb.close()
```

```
def view_lab():
```

```
    import Viewlab
```

```
ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
```

```
command=view_lab)
```

```
ViewButton.place(x=1000, y=30)
```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
```

```
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='Laboratory-Details', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='LabName:', font=('Arial', 20))
```

```
heading.place(x=350, y=200)
```

```
ln=Entry(ws, font=('Arial', 16), width=18)
```

```
ln.place(x=550,y=200)
```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
```

```
heading.place(x=350, y=300)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
```

```
dept.place(x=550,y=300)
```

```
heading=Label(ws, text='Description:', font=('Arial', 20))
heading.place(x=350, y=400)
```

```
desc=Text(ws, height = 5, width = 52)
desc.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
```

```
RegButton.place(x=550, y=500)
```

```
ws.mainloop()
```

View Labs

```
import tkinter as tk
from tkinter import ttk
import mysql.connector
```

```
def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT SNo,LabName,Department,Description FROM labs")
    records = mycursor.fetchall()
    print(records)

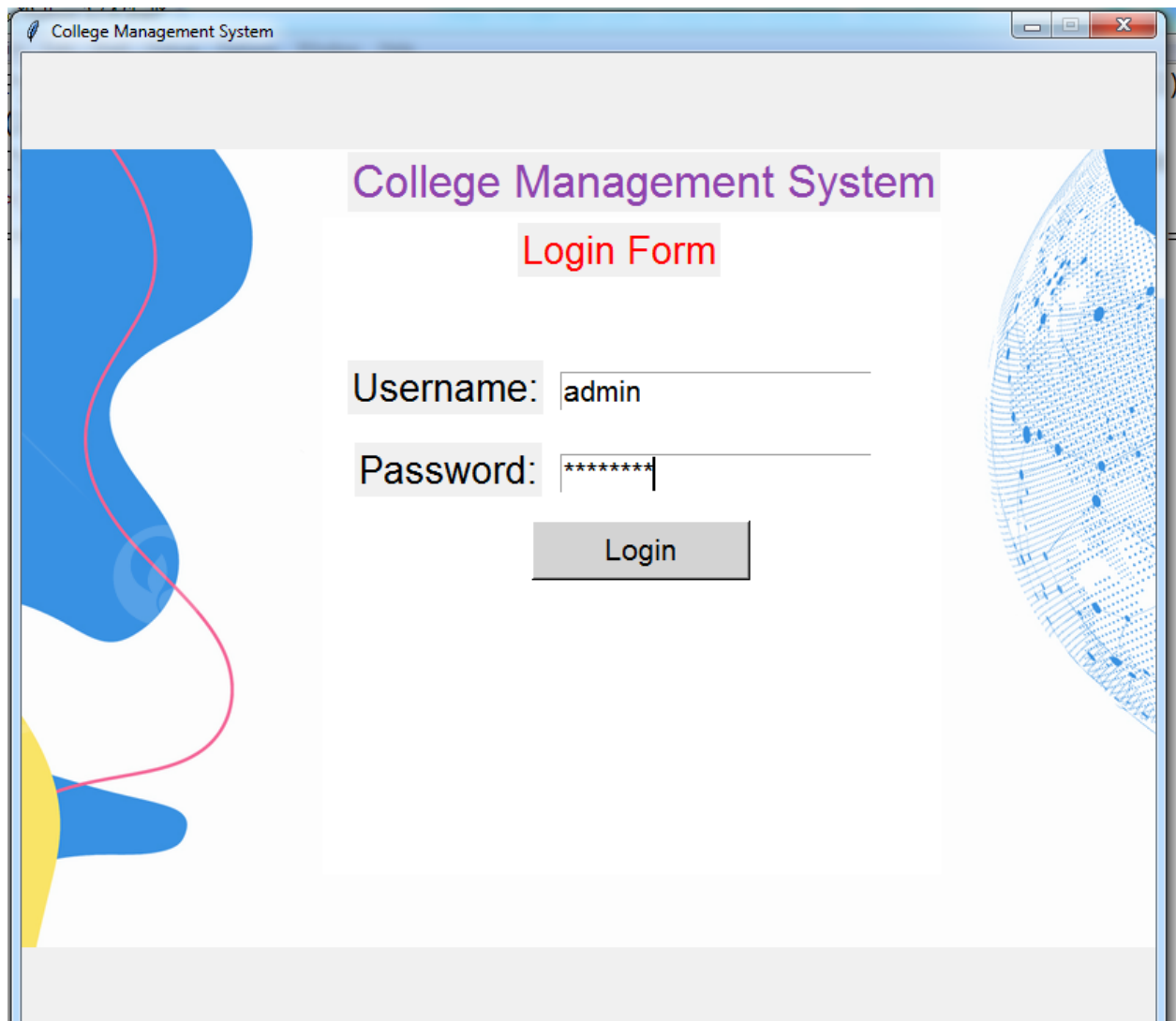
    for i, (SNo,LabName,Department,Description) in enumerate(records, start=1):
```

```
listBox.insert("", "end", values=(SNo,LabName,Department,Description))  
mysqldb.close()
```

```
root = tk.Tk()  
root.title("Lab Details")  
label = tk.Label(root, text="View Fee Details", font=("Arial",24)).grid(row=0, columnspan=3)  
  
cols = ('SNo','LabName','Department','Description')  
listBox = ttk.Treeview(root, columns=cols, show='headings')  
  
for col in cols:  
    listBox.heading(col, text=col)  
    listBox.grid(row=1, column=0, columnspan=1)  
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)  
show()  
root.mainloop()
```

Screenshots

Login



The screenshot shows a web browser window titled "College Management System". The page has a light gray header and footer. The main content area features a blue and yellow abstract graphic on the left and a blue grid pattern on the right. In the center, there is a white box containing the text "College Management System" in purple and "Login Form" in red. Below this, there are two input fields: "Username:" with the value "admin" and "Password:" with the value "*****". A gray "Login" button is positioned below the password field.

College Management System

College Management System

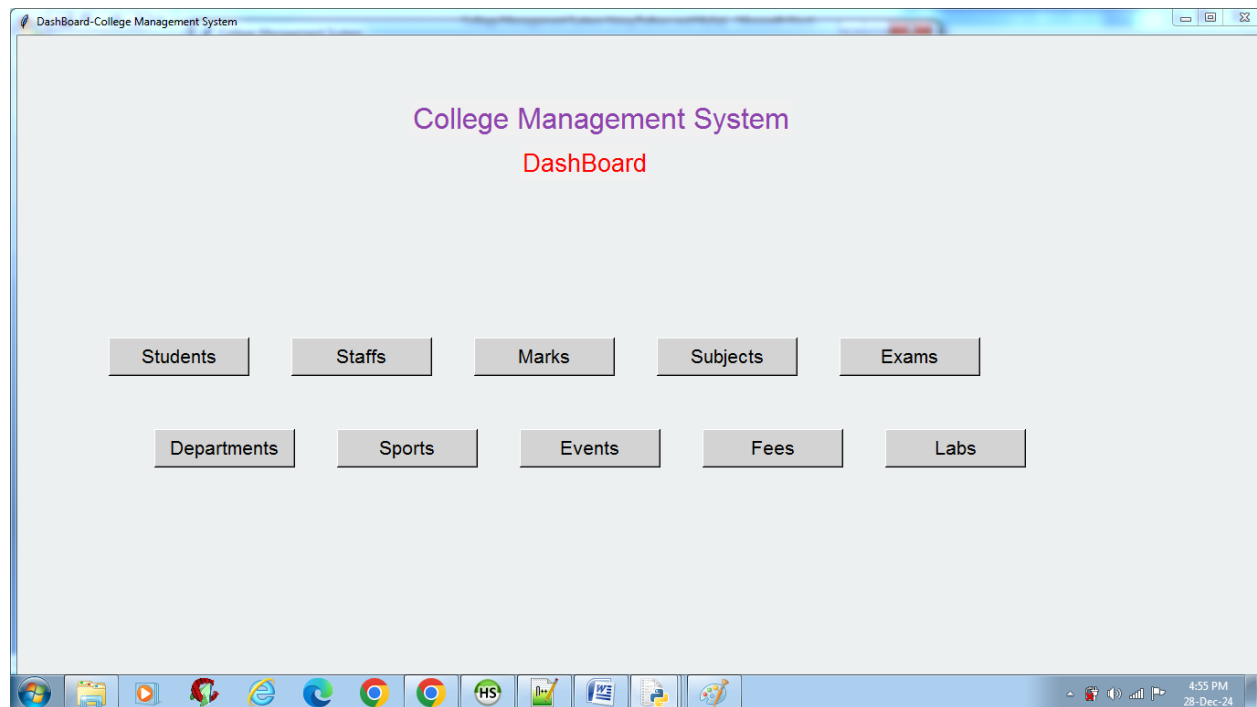
Login Form

Username: admin

Password: *****

Login

Dashboard



Students

DashBoard-College Management System

Students-College Management System

View

College Management System

Student Details

First Name: Last Name:

EmailID: Mobile:

DOB: Gender: ☒ Male ☐ Female

Class: Reg No:

Submit

View Students

College Management System

Student Details

View

Student Records

FName	LName	Email	Mobile	DOB	Gender	SClass
Ravi	S	ravi@gmail.com	9876543210	30-12-1993	Male	BSc
Siva	D	siva@gmail.com	9876543210	22-10-1999	Female	CSE
Samir	A	shamir@gmail.com	9876543210	10-12-2007	Female	BE-CSE

Class: Reg No:

Submit

Page: 80

Staff

Dashboard-College Management System

Staffs-College Management System

View

College Management System

Staff Details

Name: Qualification:

EmailID: Mobile:

Salary: Gender: ☐ Male ☒ Female

Submit

View Staff

College Management System Using Python and MySQL - Microsoft Word

Staffs-College Management System

Staff Records

Name	Qualification	Email	Mobile	Salary	Gender
Rajiv	PhD	Rajiv@gmail.com	9876543210	120000	Male

EmailID: Mobile:

Salary: Gender: ☐ Male ☒ Female

Submit

Marks

DashBoard-College Management System

Marks-College Management System

View

College Management System

Marks Details

Reg No: Name:

Subject 1: Subject 2:

Subject 3: Subject 4:

Subject 5: Subject 6:

Submit

View Marks

College Management System Using Python and MySQL - Microsoft Word

Marks-College Management System

View

Mark Details

View Mark Details

RegNo	Name	S1	S2	S3	S4	S5
10503565	Siva	78	56	34	87	87

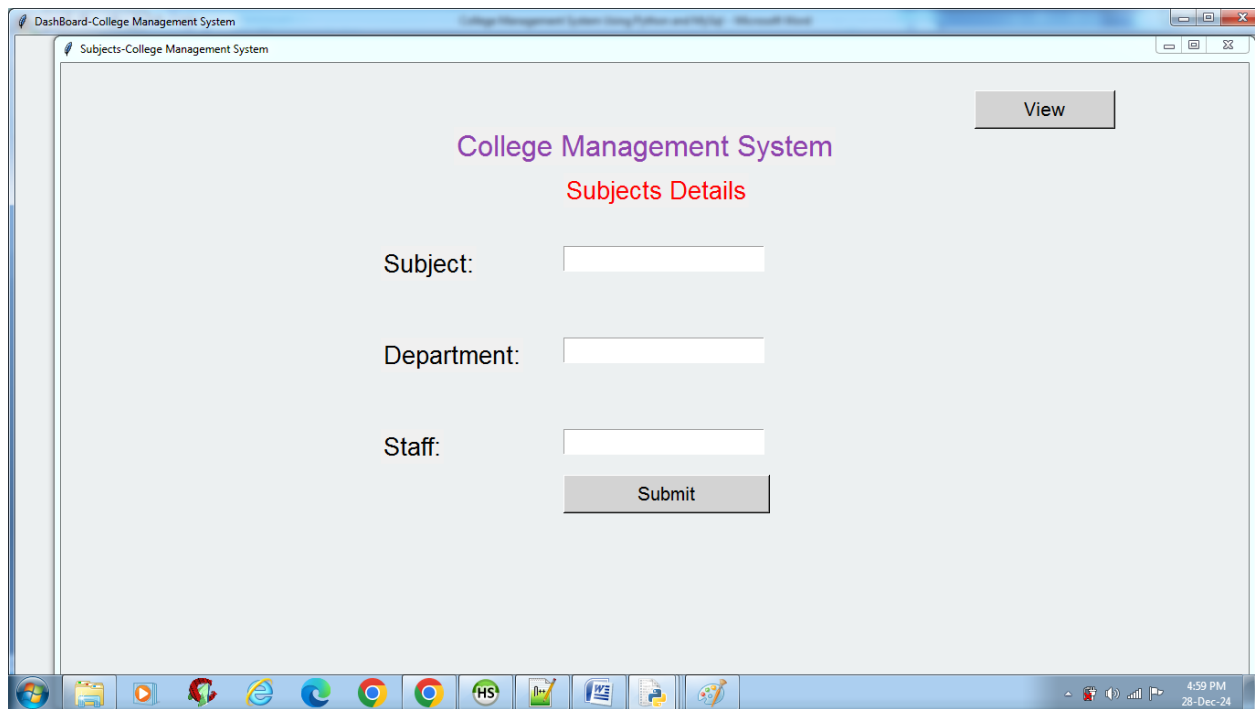
Subject 3: Subject 4:

Subject 5: Subject 6:

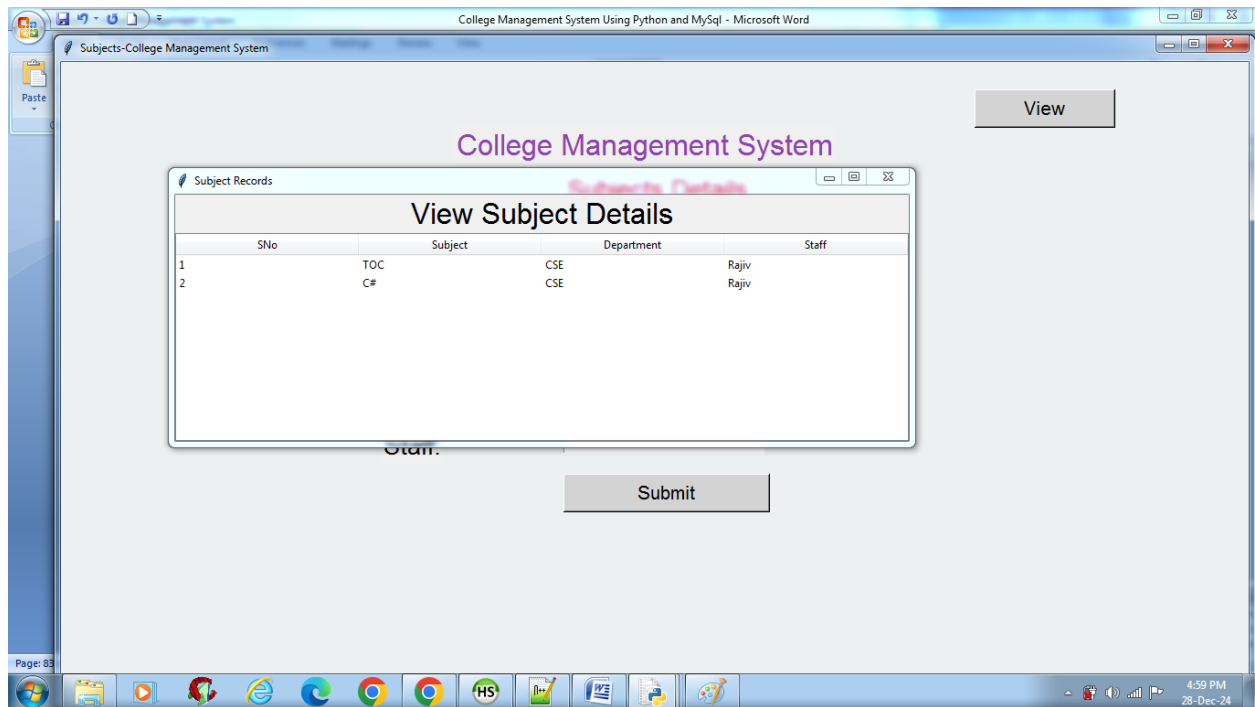
Submit

Page: 82

Subjects



View Subjects



Exams

Dashboard-College Management System

Exams-College Management System

View

College Management System

Exam Date & Details

Date: 12/28/24

Department:

Subject:

Semester:

Submit

1
2
3
4
5
6

5:00 PM
28-Dec-24

View Exam

College Management System Using Python and MySQL - Microsoft Word

Exam Records

View Exam Details

EDate	Department	Subject	Semester
1/2/25	CSE	TOC	3

View

Department:

Subject:

Semester:

Submit

5:00 PM
28-Dec-24

Departments

Dashboard-College Management System

Department-College Management System

View

College Management System

Department Details

Degree:

Department:

Description:

Submit

Windows taskbar: 5:00 PM 28-Dec-24

View All Departments

Dashboard-College Management System

Department Details

View

View Department Details

SNo	Degree	Department	Description
4	BE	CSE	Computer Science

Department:

Description:

Submit

Windows taskbar: 5:02 PM 28-Dec-24

Sports

The screenshot shows a web browser window titled 'DashBoard-College Management System'. The main content area is titled 'College Management System' in purple and 'Sports Details' in red. On the right, there is a 'View' button. The form contains five input fields: 'Sports:', 'Age:', 'Hight:', 'Weight:', and 'Description:'. Below these fields is a 'Submit' button. The browser's taskbar at the bottom shows various icons and the system clock indicating 5:02 PM on 28-Dec-24.

View

College Management System

Sports Details

Sports:

Age:

Hight:

Weight:

Description:

Submit

View Details

The screenshot shows a web browser window titled 'College Management System Using Python and MySQL - Microsoft Word'. The main content area is titled 'View Sports Details' in a grey box. On the right, there is a 'View' button. Below the title box is a table with the following data:

Sports	Age	Height	Weight	Description
Cricket	17	6	50	All Clear

Below the table, there are three input fields: 'Hight:', 'Weight:', and 'Description:'. Below these fields is a 'Submit' button. The browser's taskbar at the bottom shows various icons and the system clock indicating 5:02 PM on 28-Dec-24.

View

View Sports Details

Sports	Age	Height	Weight	Description
Cricket	17	6	50	All Clear

Hight:

Weight:

Description:

Submit

Events

Dashboard-College Management System

Events-College Management System

View

College Management System

Event Date & Place

Date: 12/28/24

EventName:

Place:

Description:

Submit

Windows taskbar: 5:03 PM 28-Dec-24

View Events

College Management System Using Python and MySQL - Microsoft Word

Events-College Management System

View

Event Records

EDate	Event	Place	Description
1/5/25	Workshop	Lab 2	For All IT Students

Description:

Submit

Windows taskbar: 5:03 PM 28-Dec-24

Fee Details

Dashboard-College Management System

College Management System Using Python and MySQL - Microsoft Word

Fees Details-College Management System

View

College Management System

Fee Details / Year

Degree:

Department:

TutionFee:

LabFee:

Stationary:

Submit

5:03 PM
28-Dec-24

View Fee Details

College Management System Using Python and MySQL - Microsoft Word

Fees Details-College Management System

View

College Management System

Fees Details

View Fee Details

SNo	Degree	Department	TutionFee	LabFee	SFee	Total
2	BE	CSE	20000	6000	2500	28500

Stationary:

Submit

Page: 88

5:03 PM
28-Dec-24

Labs

Dashboard-College Management System

Laboratory-College Management System

View

College Management System

Laboratory-Details

LabName:

Department:

Description:

Submit

5:04 PM
28-Dec-24

View Labs

College Management System Using Python and MySQL - Microsoft Word

Lab Details

View Fee Details

SNo	LabName	Department	Description
1	CSE LAB	CS & IT	Consist of 100 Computers and 2 Serv

View

Department:

Description:

Submit

Page: 89

5:04 PM
28-Dec-24